Performance Evaluation of Machine Learning Algorithms for Encrypted Proxy Traffic Identification

ZhangXiaoyuan (张校源)
Independent Researcher
Email: silenceyuan2018@gmail.com
ORCID: 0009-0006-6147-9256

***Abstract*—Encrypted proxy traffic is difficult to distinguish from other encrypted network workloads because payload inspection is unavailable. This paper evaluates a reproducible flow-level pipeline that uses 30 flow statistics for JA4Tor and 20 window statistics for MineShark and compares Random Forest (RF), support vector machine (SVM), and k-nearest neighbors (KNN) classifiers. On the original JA4Tor evaluation, RF reaches 92.12% accuracy; a capture-disjoint MineShark split gives 97.27% with SVM. Under a workload-shift experiment from NBMiner training captures to XMRig test captures, performance drops to 46.66% for SVM, showing that source and workload dependence must be reported. We provide feature-importance analysis, an auditable data split, figures, scripts, and metadata for reproduction.**

***Index Terms*—encrypted traffic, proxy detection, machine learning, network flows, reproducibility**

I. Introduction

Encrypted tunnels protect content but also limit conventional payload-based inspection. Statistical flow features offer a practical alternative for traffic classification, yet reported accuracy can be inflated by random row splits, duplicated captures, or ambiguous labels. This study focuses on an auditable baseline for encrypted proxy traffic identification and explicitly separates within-dataset performance from capture-disjoint and workload-shift evaluation.

II. Data Sources and Label Audit

The experiments use two public sources. JA4Tor provides labeled flow records for Tor and non-Tor traffic. Its NonTor label is retained as provided and is not reinterpreted as ordinary HTTPS. MineShark contains labeled cryptocurrency-mining and proxy-related captures. In particular, the dataset label `ssr` denotes ShadowsocksR, while `v2ray` denotes VMess. The available public labels do not constitute a complete four-class benchmark covering Shadowsocks, VLESS, Trojan, and ordinary HTTPS; therefore, no such four-class claim is made here.

The official ISCXVPN2016 artifact provides OpenVPN/non-VPN and application labels, which do not establish Shadowsocks, VLESS, or Trojan labels [1]. We therefore use documented artifacts from NESTTOR/JA4Tor and MineShark [3], [4]; their experiments are not pooled. JA4Tor comprises NonTor, Tor, Tor-over-Shadowsocks, Tor-over-Trojan, and Tor-over-VMess. Training contains 20,000 flows (4,000 per class), the primary test contains 5,000 (1,000 per class), and the second file contains 5,024. The public files lack complete row-to-capture provenance, preventing independent verification of capture-disjointness. The second test file is used as supplied; its acquisition domain is not inferred.

MineShark contributes 103 ShadowsocksR, 41 Trojan, and 64 VMess capture-derived records, yielding 12,401 eligible windows before sampling. Its archive MD5 is `6c64268ec35414fc9f483a7a4ed43755`. The supplied normal.pcap lacks per-flow HTTPS ground truth and is not relabeled as ordinary HTTPS. Capture files are divided approximately 70/30 within each class before balanced window sampling. IPs, ports, pool names, and acquisition dates are excluded from classifier inputs.

III. Methodology

JA4Tor uses 30 statistics derived from packet lengths, directions, counts, byte totals, and inter-arrival times (IAT). MineShark uses 20 statistics from non-overlapping 60-second windows containing at least 12 frames. Examples include mean and standard deviation of forward and backward packet lengths, maximum packet lengths, total bytes, packet counts, IAT summaries, and the down/up byte or packet ratio. Non-finite values are treated as missing and median-imputed using training data only, and numeric features are standardized for SVM and KNN. The pipeline in Fig. 1 reads PCAP/CSV input, extracts dataset-specific features, creates a documented split, trains RF/SVM/KNN, and reports classification metrics.

A. Feature Definitions and Model Settings

JA4Tor includes directional packet/byte counts, packet-length summaries, flow duration and rates, and flow and directional IAT means and extrema. MineShark includes frame-length mean, standard deviation, median, and 90th percentile; IAT mean, standard deviation, and 90th percentile; upstream frame fraction; downstream/upstream byte ratio; and direction-switching rate. For arrival times t_i, $\mathrm{IAT}_i = t_i - t_{i-1}$; for byte counts B_d, B_u, the directional byte ratio is $B_d / \max(B_u, 1)$. MineShark frame lengths include link-layer overhead and possibly oversized offloaded

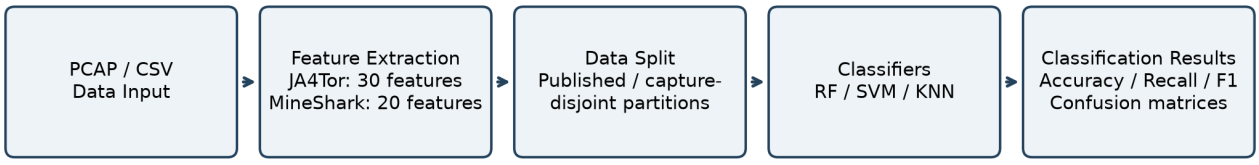


Fig. 1. Data processing and classifier evaluation pipeline.

frames, and must not be interpreted as pure TCP payload sizes. Client direction is inferred from private addresses in preprocessing; addresses are excluded from classifier inputs.

RF uses 200 trees, square-root feature sampling, minimum leaf size 2, and random seed 42. The RBF SVM uses $C = 10$ and gamma=scale; KNN uses $k = 7$. Imputation and standardization are fitted on training data only. No test-set tuning is performed. These classifiers represent ensemble trees, nonlinear margins, and distance learning [5], [6], [7], [8]. Accuracy is the fraction of correct predictions; class recall is $TP/(TP + FN)$, and macro recall averages class recalls. Balanced test classes make accuracy and macro recall equal in some results.

IV. Experimental Design and Reproducibility

For JA4Tor, we report the original test split and the separately published second test file. For MineShark, the primary split is capture-disjoint: training and test captures do not overlap. We report accuracy, macro recall, and macro F1. All saved predictions were re-evaluated independently, and the resulting metrics matched the stored CSV values. JA4Tor input hashes and the MineShark archive checksum are recorded in the repository metadata.

V. Results

A. JA4Tor

Table I summarizes the baseline and second-test-file results. RF recall is 88.60% for Tor-SS and 93.40% for Tor-Trojan. Differences on the second file cannot be directly attributed to cross-node or cross-time generalization without capture-level provenance.

B. MineShark Capture-Disjoint Evaluation

SVM recalls for ShadowsocksR, Trojan, and VMess are 98.60%, 96.51%, and 96.68%. Comparisons with JA4Tor must account for different labels, units, and workloads.

TABLE I
JA4Tor results (percent).

Test	Model	Acc.	Recall	F1
Original	RF	92.12	92.12	92.27
Original	SVM	89.72	89.72	89.90
Original	KNN	88.24	88.24	88.21
Second	RF	87.54	87.69	87.33
Second	SVM	76.73	76.99	74.38
Second	KNN	79.40	79.58	78.23

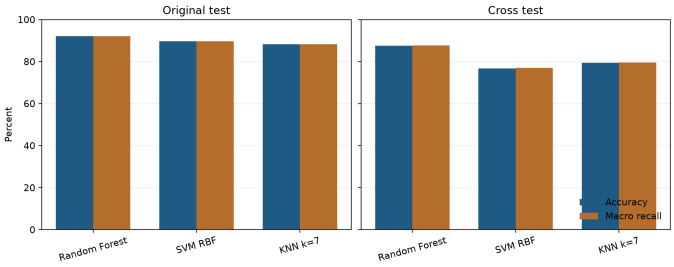


Fig. 2. JA4Tor accuracy and macro recall.

C. Workload Shift

When NBMiner captures are used for training and XMRig captures for testing, RF, SVM, and KNN obtain 44.81%, 46.66%, and 55.34% accuracy, respectively. This experiment uses 3,702 training and 1,899 test windows. RF, SVM, and KNN macro F1 are 46.08%, 42.87%, and 54.55%, respectively. SVM loses 50.61 percentage points relative to capture-disjoint evaluation. KNN superiority is limited to this fixed setting. This degradation indicates that workload and capture conditions can dominate the apparent decision boundary.

D. Feature Importance

The RF mean-decrease-in-impurity ranking identifies the following leading features on JA4Tor: Bwd Packet Length Max (8.19%), Fwd Packet Length Std (6.97%), and Down/Up Ratio (6.79%), followed by additional length

TABLE II
MineShark capture-disjoint results (percent).

Model	Train	Test	Acc.	Recall	F1
RF	4743	1719	94.30	94.30	94.30
SVM	4743	1719	97.27	97.27	97.27
KNN	4743	1719	92.44	92.44	92.43

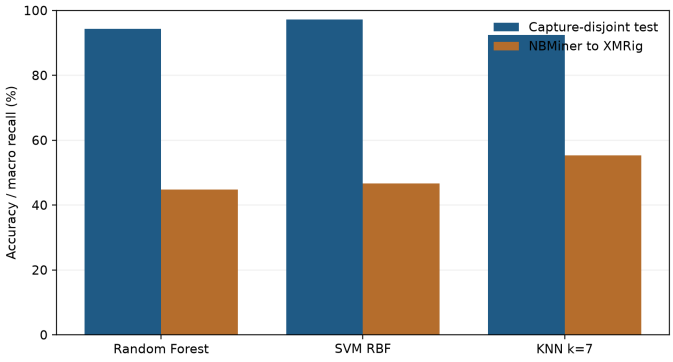


Fig. 3. Workload-shift performance from NBMiner to XMRig.

and IAT statistics. These features describe directional burst size and asymmetry, giving an interpretable indication of what the classifier uses. The identical RF configuration was refitted on 20,000 training flows and reproduced 92.12% test accuracy. The top ten account for 57.18% of importance. MDI can favor continuous or high-cardinality variables, while correlated variables share importance; the ranking is neither causal nor guaranteed stable across networks.

VI. Discussion and Limitations

The results support statistical flow classification as a useful baseline, but they also show why split design matters. A random row split can benefit from correlated flows from the same capture, while capture-disjoint and workload-shift tests are more demanding. The public sources do not provide complete and consistent labels for all proxy families requested in a four-class taxonomy. Consequently, the findings should be interpreted as benchmark-specific evidence rather than universal protocol identification.

Proxy and application behavior, network path, server, capture year, client implementation, and TLS transport can confound statistical fingerprints [2]. Future four-class evaluation should use matched browsing tasks, multiple independent nodes and capture rounds, saved PCAP hashes and protocol configurations, and node/run-disjoint partitions. Unknown proxies and padding/multiplexing should also be tested.

VII. Conclusion

We presented an IEEE-style, reproducible pipeline for encrypted proxy traffic classification using 30 flow statistics and three classical classifiers. RF performs strongly

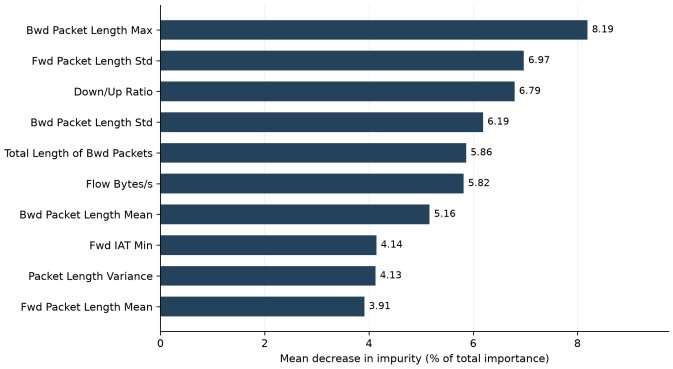


Fig. 4. Top RF feature-importance ranking.

on the original JA4Tor split, SVM performs best on the capture-disjoint MineShark split, and all methods degrade under workload shift. Feature ranking and explicit label auditing improve interpretability and prevent overclaiming.

VIII. Code and Data Availability

Scripts, result files, figures, metadata, and the editable paper source are available at:
https://github.com/silenceyuana/encrypted-proxy-traffic-identification.

IX. Generative AI Usage Disclosure

OpenAI Codex assisted with manuscript drafting, Chinese/English expression and polishing, experiment-code development, figure creation, and layout checking. Training, evaluation, and feature-importance values were computed by local Python/scikit-learn programs; AI-generated prose is not an experimental data source. The author is responsible for reviewing all text, references, code, and results before submission and for the final manuscript accuracy, originality, and research integrity.

Appendix

The recorded environment is Python 3.12 and scikit-learn 1.9.1. The repository scripts `run_ja4tor_baseline.py`, `run_mineshark_proxy_baseline.py`, and `run_mineshark_workload_shift.py` reproduce the three experiments. `plot_workload_shift.py` generates the comparison plot, and `analyze_feature_importance.py` regenerates feature rankings and the methodology figure. Run each with Python from the project root using the `scripts/` prefix. Experiment manifests and capture-split CSVs record versions, hashes, feature definitions, and partitions. Original third-party data are downloaded separately; raw archives are not redistributed. Data sources: https://github.com/das-lab/JA4Tor and https://zenodo.org/records/13624058.

References

[1] G. Draper-Gil et al., "Characterization of encrypted and VPN traffic using time-related features," in *Proc. ICISSP*, 2016, pp. 407–414. [Online]. Available: https://www.unb.ca/cic/datasets/vpn.html

[2] D. Xue et al., "Fingerprinting obfuscated proxy traffic with encapsulated TLS handshakes," in *Proc. 33rd USENIX Security Symp.*, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/xue-fingerprinting

[3] C. Zhou et al., "NESTTOR: Detecting Tor-over-Proxy traffic with factorized decisions," *IEEE Trans. Netw. Service Manag.*, 2026, doi: 10.1109/TNSM.2026.3743261.

[4] S. Xi et al., "MineShark: Cryptomining traffic detection at scale," in *Proc. NDSS*, 2025, doi: 10.14722/ndss.2025.230402.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[6] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995, doi: 10.1007/BF00994018.

[7] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967, doi: 10.1109/TIT.1967.1053964.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.